

错误处理规范

2026年03月14日 v1.0

错误处理规范

文档版本：v1.0

编写日期：2026 年 3 月 14 日

开发负责人：王舟

一、错误码规范

1.1 错误码结构

错误码 = 模块码 (2 位) + 错误类型 (2 位) + 具体错误 (2 位)

示例：400101

- 40：认证模块
- 01：验证错误
- 01：用户名或密码错误

1.2 全局错误码

错误码	错误名	HTTP 状态	说明
200000	SUCCESS	200	成功
400000	BAD_REQUEST	400	请求参数错误
400001	VALIDATION_ERROR	400	数据验证失败
401000	UNAUTHORIZED	401	未认证

错误码	错误名	HTTP 状态	说明
401001	TOKEN_INVALID	401	Token 无效
401002	TOKEN_EXPIRED	401	Token 过期
403000	FORBIDDEN	403	无权限
404000	NOT_FOUND	404	资源不存在
500000	INTERNAL_ERROR	500	服务器内部错误
500001	DATABASE_ERROR	500	数据库错误

1.3 模块错误码

认证模块 (40)

错误码	错误名	说明
400101	AUTH_INVALID_CREDENTIALS	用户名或密码错误
400102	AUTH_USER_NOT_FOUND	用户不存在
400103	AUTH_USER_INACTIVE	用户已停用
400104	AUTH_TOO_MANY_ATTEMPTS	登录尝试次数过多
400105	AUTH_PASSWORD_WEAK	密码强度不足

案件模块 (41)

错误码	错误名	说明
410101	CASE_NOT_FOUND	案件不存在

错误码	错误名	说明
410102	CASE_NO_PERMISSION	无案件访问权限
410103	CASE_ALREADY_DELETED	案件已删除

文件模块 (42)

错误码	错误名	说明
420101	FILE_NOT_FOUND	文件不存在
420102	FILE_TOO_LARGE	文件过大
420103	FILE_TYPE_NOT_ALLOWED	文件类型不支持
420104	FILE_PARSE_FAILED	文件解析失败

时间轴模块 (43)

错误码	错误名	说明
430101	NODE_NOT_FOUND	节点不存在
430102	NODE_TIME_CONFLICT	时间冲突

人物模块 (44)

错误码	错误名	说明
440101	PERSON_NOT_FOUND	人物不存在
440102	PERSON_ALREADY_EXISTS	人物已存在

二、Rust 错误处理

2.1 错误类型定义

```
// src/errors.rs
use axum::{
    http::StatusCode,
    response::{IntoResponse, Response},
    Json,
};
use serde::{Deserialize, Serialize};
use thiserror::Error;

/// 应用错误
#[derive(Debug, Error)]
pub enum AppError {
    #[error("请求参数错误: {0}")]
    BadRequest(String),

    #[error("验证失败: {0}")]
    ValidationError(String),

    #[error("未认证")]
    Unauthorized,

    #[error("Token 无效")]
    InvalidToken,

    #[error("Token 过期")]
    TokenExpired,

    #[error("无权限")]
    Forbidden,

    #[error("资源不存在: {0}")]
    NotFound(String),

    #[error("内部错误: {0}")]
    Internal(String),

    #[error("数据库错误: {0}")]
    Database(String),

    #[error("文件错误: {0}")]
    File(String),
}
```

```

}

/// 错误响应
#[derive(Debug, Serialize, Deserialize)]
pub struct ErrorResponse {
    pub code: i32,
    pub message: String,
    pub details: Option<serde_json::Value>,
    pub timestamp: i64,
}

impl IntoResponse for AppError {
    fn into_response(self) -> Response {
        let (status, error_code, message) = match &self {
            AppError::BadRequest(msg) => (StatusCode::BAD_REQUEST, 400000, msg),
            AppError::ValidationError(msg) => (StatusCode::BAD_REQUEST, 400001, msg),
            AppError::Unauthorized => (StatusCode::UNAUTHORIZED, 401000, "未认证"),
            AppError::InvalidToken => (StatusCode::UNAUTHORIZED, 401001, "Token 无效"),
            AppError::TokenExpired => (StatusCode::UNAUTHORIZED, 401002, "Token 过期"),
            AppError::Forbidden => (StatusCode::FORBIDDEN, 403000, "无权限"),
            AppError::NotFound(msg) => (StatusCode::NOT_FOUND, 404000, msg),
            AppError::Database(msg) => (StatusCode::INTERNAL_SERVER_ERROR, 500001, msg),
            AppError::File(msg) => (StatusCode::BAD_REQUEST, 420000, msg),
            AppError::Internal(msg) => (StatusCode::INTERNAL_SERVER_ERROR, 500000, msg),
        };

        let error_response = ErrorResponse {
            code: error_code,
            message: message.to_string(),
            details: None,
            timestamp: chrono::Utc::now().timestamp(),
        };

        (status, Json(error_response)).into_response()
    }
}

/// 统一错误处理宏
#[macro_export]
macro_rules! ensure {
    ($cond:expr, $err:expr) => {
        if !($cond) {
            return Err($err);
        }
    };
}

```

```
/// 结果类型别名
pub type Result<T> = std::result::Result<T, AppError>;
```

2.2 错误处理示例

```
// src/routers/cases.rs
use crate::errors::{AppError, Result};
use crate::models::case::Case;

/// 获取案件详情
pub async fn get_case(
    State(state): State<Arc<AppState>>,
    Path(case_id): Path<String>,
    claims: Claims,
) -> Result<Json<Case>> {
    // 检查案件是否存在
    let case = state.db
        .get_case(&case_id)
        .await
        .map_err(|e| AppError::Database(e.to_string()))?
        .ok_or_else(|| AppError::NotFound(format!("案件 {} 不存在", case_id)))?;

    // 检查权限
    let has_permission = state.auth_service
        .check_permission(&claims.sub, &case_id, "case:view")
        .await
        .map_err(|e| AppError::Internal(e.to_string()))?;

    ensure!(has_permission, AppError::Forbidden);

    Ok(Json(case))
}

/// 创建案件
pub async fn create_case(
    State(state): State<Arc<AppState>>,
    claims: Claims,
    Json(input): Json<CreateCaseInput>,
) -> Result<Json<Case>> {
    // 验证输入
    input.validate().map_err(AppError::ValidationError)?;

    // 检查案件名称是否重复
    let existing = state.db
        .find_case_by_name(&claims.sub, &input.name)
```

```

        .await
        .map_err(|e| AppError::Database(e.to_string()))?;

    ensure!(
        existing.is_none(),
        AppError::BadRequest("案件名称已存在".to_string())
    );

    // 创建案件
    let case = state.db
        .create_case(&claims.sub, &input)
        .await
        .map_err(|e| AppError::Database(e.to_string()))?;

    // 记录操作日志
    log_operation!(
        claims,
        action: ActionType::Create,
        module: ModuleType::Case,
        target_type: "case",
        target_id: case.id.clone(),
        target_title: Some(case.name.clone()),
        old: None,
        new: Some(json!(&case))
    );

    Ok(Json(case))
}

```

2.3 错误日志

```

// src/middleware/error_logger.rs
use axum::http::StatusCode;
use tracing::{error, warn, info};

pub async fn log_error(err: AppError) {
    match &err {
        AppError::BadRequest(msg) | AppError::ValidationError(msg) => {
            warn!("客户端错误: {}", msg);
        }
        AppError::Unauthorized | AppError::InvalidToken | AppError::Forbidden => {
            warn!("认证/授权错误: {}", err);
        }
        AppError::NotFound(msg) => {
            info!("资源不存在: {}", msg);
        }
    }
}

```

```

    }
    AppError::Database(msg) | AppError::Internal(msg) => {
        error!("服务器错误: {}", msg);
    }
    _ => {
        error!("未知错误: {}", err);
    }
}
}

```

三、前端错误处理

3.1 错误处理 Hook

```

// frontend/src/hooks/useErrorHandler.rs

pub enum ErrorType {
    Network,
    Auth,
    Validation,
    Server,
    Unknown,
}

pub fn use_error_handler() -> ErrorHandler {
    ErrorHandler::new()
}

pub struct ErrorHandler;

impl ErrorHandler {
    pub fn new() -> Self {
        Self
    }

    pub fn handle(&self, error: &ApiError) {
        match error {
            ApiError::Network(msg) => {
                toast::error("网络错误", msg);
            }
            ApiError::Auth(code, msg) => {

```



```

        match code {
            401001 | 401002 => {
                // Token 无效/过期, 跳转到登录页
                toast::warning("登录已过期", "请重新登录");
                clear_auth();
                navigate_to("/login");
            }
            403000 => {
                toast::error("无权限", msg);
            }
            _ => {
                toast::error("认证错误", msg);
            }
        }
    }

    ApiError::Validation(errors) => {
        // 显示表单验证错误
        for (field, message) in errors {
            form_error::set(field, message);
        }
        toast::warning("数据验证失败", "请检查表单输入");
    }

    ApiError::Server(code, msg) => {
        toast::error("服务器错误", msg);
        // 记录错误日志
        log_error("server_error", code, msg);
    }

    ApiError::Unknown(msg) => {
        toast::error("未知错误", msg);
    }
}

pub fn is_auth_error(&self, error: &ApiError) -> bool {
    matches!(error, ApiError::Auth(401001 | 401002, _))
}
}

```

3.2 API 请求错误处理

```

// frontend/src/api/mod.rs

pub async fn request<T: DeserializeOwned>(
    method: Method,
    path: &str,

```

```

    body: Option<impl Serialize>,
) -> Result<T, ApiError> {
    let token = get_access_token();

    let mut request = Request::builder()
        .method(method)
        .uri(format!("/api/v1{}", path));

    if let Some(t) = &token {
        request = request.header("Authorization", format!("Bearer {}", t));
    }

    if let Some(b) = body {
        request = request.header("Content-Type", "application/json");
        request = request.body(serde_json::to_string(&b).unwrap());
    }

    let response = fetch(request).await
        .map_err(|e| ApiError::Network(e.to_string()))?;

    // 处理错误响应
    if !response.status().is_success() {
        let error_response: ErrorResponse = response.json().await
            .map_err(|_| ApiError::Unknown("无法解析错误响应".to_string()))?;

        return Err(ApiError::from_error_response(error_response));
    }

    let data: T = response.json().await
        .map_err(|e| ApiError::Unknown(e.to_string()))?;

    Ok(data)
}

/// API 错误
#[derive(Debug, Clone)]
pub enum ApiError {
    Network(String),
    Auth(i32, String),
    Validation(HashMap<String, String>),
    Server(i32, String),
    NotFound(i32, String),
    Unknown(String),
}

impl ApiError {
    pub fn from_error_response(error: ErrorResponse) -> Self {

```

```

match error.code {
  401000..=401999 => ApiError::Auth(error.code, error.message),
  400001 => ApiError::Validation(
    error.details
      .and_then(|d| d.as_object().cloned())
      .map(|obj| {
        obj.into_iter()
          .map(|(k, v)| (k, v.as_str().unwrap_or(").to_string()))
          .collect()
      })
      .unwrap_or_default()
  ),
  404000 => ApiError::NotFound(error.code, error.message),
  500000..=599999 => ApiError::Server(error.code, error.message),
  _ => ApiError::Unknown(error.message),
}
}
}

```

3.3 错误边界组件

```

// frontend/src/components/ErrorBoundary.rs

#[component]
pub fn ErrorBoundary<F, C>(fallback: F, children: C) -> Element
where
  F: FnOnce(ErrorBoundaryContext) -> Element,
  C: Component,
{
  let mut error = use_signal(|| None::<ErrorBoundaryContext>);

  // 捕获子组件错误
  use_context_provider(|| error);

  if let Some(err) = error() {
    fallback(err)
  } else {
    children()
  }
}

/// 错误边界上下文
#[derive(Clone, Copy)]
pub struct ErrorBoundaryContext {
  pub error: String,
}

```

```
pub component: String,
}

// 使用示例
#[component]
pub fn App() -> Element {
    rsx! {
        ErrorBoundary {
            fallback: |ctx| rsx! {
                div { class: "error-page",
                    h1 { "出错了" }
                    p { "{ctx.error}" }
                    button { onclick: move |_| location::reload(), "刷新页面" }
                }
            },
            Router::<Route> {}
        }
    }
}
```

四、用户友好提示

4.1 错误提示文案

错误类型	技术文案	用户文案
401001	Token 无效	登录已过期，请重新登录
401002	Token 过期	登录已过期，请重新登录
403000	无权限	您没有访问此内容的权限
404000	资源不存在	该内容不存在或已被删除
420102	文件过大	文件大小不能超过 100MB
420103	文件类型不支持	不支持该文件格式

错误类型	技术文案	用户文案
420104	文件解析失败	文件解析失败，请检查文件是否损坏
500000	内部错误	服务器开小差了，请稍后重试
500001	数据库错误	数据保存失败，请稍后重试

4.2 Toast 提示组件

```
// frontend/src/components/Toast.rs

#[derive(Clone, Copy, PartialEq)]
pub enum ToastType {
    Success,
    Info,
    Warning,
    Error,
}

#[component]
pub fn Toast(message: String, toast_type: ToastType) -> Element {
    let icon = match toast_type {
        ToastType::Success => "✅",
        ToastType::Info => "ℹ️",
        ToastType::Warning => "⚠️",
        ToastType::Error => "❌",
    };

    rsx! {
        div { class: "toast {toast_type}",
            span { class: "icon", {icon} }
            span { class: "message", {message} }
        }
    }
}

// 使用示例
toast::success("操作成功", "案件已创建");
toast::error("操作失败", "文件上传失败，请检查网络");
```

五、错误监控

5.1 错误日志表

```
CREATE TABLE error_logs (  
  id          TEXT PRIMARY KEY,  
  error_code  INTEGER NOT NULL,  
  error_message TEXT NOT NULL,  
  error_stack TEXT,  
  request_url TEXT,  
  request_method TEXT,  
  user_id     TEXT,  
  user_agent  TEXT,  
  ip_address  TEXT,  
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
  INDEX idx_code ON error_logs(error_code),  
  INDEX idx_time ON error_logs(created_at),  
  INDEX idx_user ON error_logs(user_id)  
);
```

5.2 错误统计

```
-- 错误统计视图  
CREATE VIEW error_stats AS  
SELECT  
  error_code,  
  error_message,  
  COUNT(*) as error_count,  
  DATE(created_at) as error_date  
FROM error_logs  
GROUP BY error_code, error_message, DATE(created_at)  
ORDER BY error_count DESC;
```

六、总结

6.1 错误处理流程



6.2 错误码清单

模块	错误码范围	数量
全局	200000-599999	10
认证	400101-400105	5
案件	410101-410103	3
文件	420101-420104	4
时间轴	430101-430102	2
人物	440101-440102	2
总计		26

文档版本：v1.0

最后更新：2026 年 3 月 14 日

开发负责人：王舟

邮箱：main@mails.wedevs.org

电话：15378391447

开发负责人：王舟 | 电话：15378391447 | 邮箱：main@mails.wedevs.org